Name bhavya rana
Sapid 590034689
Batch 15

C program

```
#include <stdio.h>

void merge_sorted_arrays(int *arr1, int n1, int *arr2, int
n2)
{
    int merged[n1 + n2];
    int i = 0, j = 0, k = 0;

    // Compare elements of both arrays
    while (i < n1 && j < n2)
    {
        if (*(arr1 + i) <= *(arr2 + j))
        {
            merged[k] = *(arr1 + i);
            i++;
        }
        else
        {
            merged[k] = *(arr2 + j);
            j++;
        }
        k++;
    }

    // Copy remaining elements of arr1
    while (i < n1)
    {
        merged[k] = *(arr1 + i);
        i++;
        k++;
    }
```

```
    // Copy remaining elements of arr2
    while (j < n2)
    {
        merged[k] = *(arr2 + j);
        j++;
        k++;
    }

    // Display merged array
    for (i = 0; i < n1 + n2; i++)
    {
        printf("%d", merged[i]);

        if (i < n1 + n2 - 1)
            printf(" ");
    }
}

int main()
{
    int n1, n2;

    printf("Enter size of Array 1: ");
    scanf("%d", &n1);

    int arr1[n1];

    printf("Enter elements of Array 1: ");
    for (int i = 0; i < n1; i++)
    {
        scanf("%d", &arr1[i]);
    }

    printf("Enter size of Array 2: ");
    scanf("%d", &n2);

    int arr2[n2];

    printf("Enter elements of Array 2: ");
    for (int i = 0; i < n2; i++)
    {
        scanf("%d", &arr2[i]);
    }

    printf("Merged array: ");
```

```
    merge_sorted_arrays(arr1, n1, arr2, n2);

    return 0;
}
```

Algorithm

1. Read `n1` and the elements of `arr1[]`.
2. Read `n2` and the elements of `arr2[]`.
3. Pass the addresses of both arrays to `merge_sorted_arrays()`.
4. Initialize two pointers/indexes `i = 0` and `j = 0` for the two arrays and `k = 0` for `merged[]`.
5. Compare `arr1[i]` and `arr2[j]`.
6. Store the smaller element in `merged[k]` and move the corresponding pointer forward.
7. If the elements are equal, selecting one first and then the other later preserves all duplicates.
8. Continue until one array is exhausted.
9. Copy the remaining elements of the other array into `merged[]`.
10. Display all elements of `merged[]` separated by spaces.

Test Case Table

Test	arr1[]	arr2[]	Expected Output
1	1 3 5 7	2 4 6 8	1 2 3 4 5 6 7 8
2	1 2 2 4	2 3 5	1 2 2 2 3 4 5
3	1 4 7	2 3 5 6 8	1 2 3 4 5 6 7 8
4	1 2 3	4 5 6	1 2 3 4 5 6
5	2 4 6	1 3 5	1 2 3 4 5 6

Execution

```
Last login: Wed Nov  4 21:34:19 on ttys000
bhavya@Bhavyas-MacBook-Air ~ % nano day62.c
bhavya@Bhavyas-MacBook-Air ~ % clang day62.c -o day62
bhavya@Bhavyas-MacBook-Air ~ % ./day62
Enter size of Array 1: 4
Enter elements of Array 1: 1 3 5 7
Enter size of Array 2:  2 4 6 8
Enter elements of Array 2: Merged array: 1 3 4 5 6 7%              bhavya@Bhavyas-MacBook-Air ~ % ./day62
Enter size of Array 1: 4
Enter elements of Array 1: 1 2 2 4
Enter size of Array 2: 2 3 5
Enter elements of Array 2: Merged array: 1 2 2 3 4 5%              bhavya@Bhavyas-MacBook-Air ~ % ./day62
Enter size of Array 1: 3
Enter elements of Array 1: 1 2 3
Enter size of Array 2: 4 5 6
Enter elements of Array 2: 4 5 6
bhavya@Bhavyas-MacBook-Air ~ %
```